

SMARTRIDE: BLUETOOTH-BASED TWO-WHEELER RIDE LOGGER AND CRASH DETECTION SYSTEM USING ESP32-S3

A PROJECT REPORT

submitted by

Adarsh Raj	(TKI23CS005)
Muhammed Afsal	(TKI23CS074)
Irfan V	(TKI23CS050)
Rojith Ram	(TKI23CS097)

to

the **APJ Abdul Kalam Technological University**

in partial fulfillment of the requirements for the award of the Degree of

Bachelor of Technology in

Computer Science and Engineering



Department of Computer Science and Engineering

TKM Institute of Technology

Karuvelil, Kollam

September 2026

DECLARATION

We undersigned hereby declare that the project report “SmartRide: Bluetooth-Based Two-Wheeler Ride Logger and Crash Detection System Using ESP32-S3”, submitted for partial fulfillment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala, is a bonafide work done by us under the supervision of Mrs. Neetha Alex.

This submission represents our ideas in our own words and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the Institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained.

This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

Place: Kollam

Adarsh Raj

Muhammed Afsal

Date:

Irfan V

Rojith Ram

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
TKM INSTITUTE OF TECHNOLOGY, KARUVELIL, KOLLAM



CERTIFICATE

This is to certify that the report entitled “**SmartRide: Bluetooth-Based Two-Wheeler Ride Logger and Crash Detection System Using ESP32-S3**” submitted by **Adarsh Raj, Muhammed Afsal, Irfan V, Rojith Ram** to the APJ Abdul Kalam Technological University in partial fulfilment of the requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering is a Bonafide record of the project carried out by them under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Mrs. Neetha Alex
Assistant Professor
Project Guide

Dr. Nijil Raj N
Head of the Department

Dr. Reji R
Professor
Project Coordinator

Mr. Rupesh Ravi M R
Assistant Professor
Project Coordinator

ACKNOWLEDGEMENT

First of all, We thank God Almighty for helping us to successfully complete our Project.

We express our sincere thanks and a deep sense of humble gratitude to the Principal of this institution, **Dr. Gouri Mohan L**, for providing all the necessary facilities.

We thank **Dr. Nijil Raj N**, Head of the Department, Computer Science & Engineering, for all the words of inspiration.

Next, We would like to thank our project coordinators **Dr. Reji R**, **Mr. Rupesh Ravi** for their valuable advices.

We have this great pleasure to express our deep sense of gratitude and obligation to our project guide **Mrs. Neetha Alex** for her valuable guidance and suggestions throughout the entire project work.

Last but certainly not the least, We would also like to thank all faculty and staff of the Department of Computer Science & Engineering and my friends for their help and co-operation.

TKI23CS005 Adarsh Raj

TKI23CS074 Muhammed Afsal

TKI23CS050 Irfan V

TKI23CS097 Rojith Ram

ABSTRACT

Two-wheelers constitute the largest share of registered vehicles on Indian roads, yet the overwhelming majority of them offer the rider no digital record of how they are ridden and no assistance when the rider falls. This project, *SmartRide: Bluetooth-Based Two-Wheeler Ride Logger and Crash Detection System Using ESP32-S3*, proposes a compact, self-contained on-board unit that logs ride data, recognises crashes from motion signatures and delivers both to the rider's smartphone over Bluetooth Low Energy, without depending on cellular connectivity or cloud services.

The on-board unit is built around an ESP32-S3 microcontroller, which reads location, speed and course from an ATGM336H-5N multi-constellation GNSS receiver over UART and acquires motion data from an MPU6500 six-axis accelerometer and gyroscope over a high-speed serial interface. An ADXL345 three-axis accelerometer is provided as an optional secondary impact channel, offering an independent hardware interrupt for corroborating high-energy events. The unit is powered from a lithium-ion cell managed by a dedicated battery management system (BMS), which enforces over-charge, over-discharge and over-current protection and allows the logger to remain alive through the shock and supply interruption of a crash.

Ride parameters such as instantaneous speed, cumulative distance, duration and travelled route are computed on the microcontroller itself and exposed as Bluetooth Low Energy GATT characteristics. Crash detection uses a multi-condition rule that combines resultant acceleration, sustained tilt derived from gyroscope integration, and post-impact stillness, so that potholes, speed breakers and kerb strikes do not raise false alarms. A Flutter Android application acts purely as a BLE client: it renders a live dashboard, plots the route, stores ride history in a local SQLite database and issues distance-based maintenance reminders from the cumulative odometer reading.

By keeping sensing, computation, crash logic and storage entirely on the device and the paired handset, SmartRide delivers an affordable, network-independent and privacy-preserving smart riding solution. The system requires no SIM card, no recurring data plan and no vendor cloud account, which makes it suitable as a low-cost retrofit for the existing two-wheeler population.

Keywords: *Internet of Things, Two-Wheeler Telematics, Ride Logging, Crash Detection, Bluetooth Low Energy, ESP32-S3, MPU6500, ATGM336H-5N, ADXL345, GNSS, Flutter, Offline Operation, Rider Safety*

MAPPING WITH PROGRAM OUTCOMES (POs)

The mapping between the project contributions and relevant POs is summarized below.

PO	Contribution of the Project
PO1: Engineering Knowledge	Applied principles of embedded systems, sensor signal processing, wireless communication and mobile application development to realise a self-contained two-wheeler ride logger.
PO2: Problem Analysis	Identified and analysed the absence of affordable ride telematics and crash notification on two-wheelers, and the dependence of existing solutions on cellular coverage.
PO3: Design/Development of Solutions	Designed a complete on-board architecture comprising the ESP32-S3, GNSS receiver, inertial sensors, battery management and a BLE-connected Android application.
PO4: Conduct Investigations of Complex Problems	Planned controlled drop, tilt and on-road trials to characterise sensor behaviour and to calibrate crash-detection thresholds against false-alarm sources.
PO5: Modern Tool Usage	Utilised ESP-IDF, the Arduino core for ESP32-S3, Flutter, SQLite, Python-based log analysis and Git for efficient development, validation and version control.
PO6: The Engineer and Society	Addresses a road-safety problem of direct public consequence by shortening the interval between a two-wheeler crash and the moment help is summoned.
PO7: Environment and Sustainability	Encourages timely, distance-based servicing that extends vehicle life and reduces emissions from poorly maintained engines, supporting SDG 11 and SDG 12.
PO9: Individual and Team Work	Enabled the team to divide firmware, application, hardware integration and testing responsibilities while working towards a single integrated deliverable.
PO10: Communication	Developed the ability to document and present technical work through interim evaluation, design reviews and this project report.
PO12: Life-long Learning	Required independent study of GNSS protocols, inertial sensing, the Bluetooth Low Energy GATT model and lithium-ion battery management beyond the prescribed syllabus.

Table 0.1: Mapping of Project with Program Outcomes (POs)

MAPPING WITH COURSE OUTCOMES (COs)

The mapping between the project objectives and relevant Course Outcomes (COs) is summarized below.

CO	Contribution of the Project
CO1: Problem Identification	Identified real-world problems in smart transportation — the absence of ride history, delayed crash response and guesswork-based servicing — and proposed an IoT-based solution. (<i>Mapped to PO1, PO2</i>)
CO2: System Design	Designed and specified an ESP32-S3 based Bluetooth ride-logging system with defined hardware interfaces, firmware tasks and a BLE GATT service. (<i>Mapped to PO3</i>)
CO3: Implementation	Integrated the ATGM336H-5N GNSS receiver, the MPU6500 inertial measurement unit and BLE communication for real-time ride monitoring. (<i>Mapped to PO5</i>)
CO4: Data Analysis	Analysed logged ride and impact data to evaluate distance accuracy, detection latency and false-alarm behaviour through field testing. (<i>Mapped to PO4</i>)
CO5: Use of Modern Tools	Applied ESP-IDF, Flutter, SQLite, the Google Maps API and Python analysis scripts for development, visualisation and threshold tuning. (<i>Mapped to PO5</i>)
CO6: Societal Benefit and Sustainability	Delivered a safety-oriented, affordable solution that benefits riders and promotes timely maintenance. (<i>Mapped to PO6, PO7</i>)
CO7: Teamwork and Communication	Worked collaboratively with clear task allocation and documented the project professionally through reviews and reports. (<i>Mapped to PO9, PO10, PO12</i>)

Table 0.2: Mapping of Project with Course Outcomes (COs)

MAPPING WITH SUSTAINABLE DEVELOPMENT GOALS (SDGs)

The project **SmartRide: Bluetooth-Based Two-Wheeler Ride Logger and Crash Detection System Using ESP32-S3** has strong alignment with the United Nations Sustainable Development Goals (SDGs). The mapping of the project to the relevant SDGs is presented below.

SDG	Contribution of the Project
SDG 3: Good Health and Well-being	Target 3.6 aims to reduce deaths and injuries from road traffic accidents. Faster crash notification shortens the interval before help reaches an injured two-wheeler rider.
SDG 9: Industry, Innovation and Infrastructure	Demonstrates affordable embedded innovation that can be retrofitted to vehicles already on the road, requiring no new roadside or network infrastructure.
SDG 11: Sustainable Cities and Communities	Target 11.2 seeks safe and accessible transport for all. Ride data and automatic safety alerts contribute directly to safer urban mobility.
SDG 12: Responsible Consumption and Production	Distance-based maintenance reminders promote timely servicing, extending vehicle life and reducing emissions and waste from poorly maintained engines.

Table 0.3: Mapping of Project with Sustainable Development Goals (SDGs)

FEASIBILITY REPORT

(i) TECHNICAL FEASIBILITY

Hardware Feasibility:

- **Microcontroller:** The ESP32-S3 provides a dual-core Xtensa LX7 processor at up to 240 MHz, ample SRAM with optional PSRAM, and an integrated Bluetooth 5 Low Energy radio — sufficient for concurrent sensor sampling, on-device computation and BLE service hosting.
- **Positioning:** The ATGM336H-5N is a low-cost multi-constellation GNSS module that outputs standard NMEA sentences over UART and provides a 1PPS timing output, giving reliable position, speed and course fixes under Indian sky conditions.
- **Motion Sensing:** The MPU6500 six-axis accelerometer and gyroscope supports selectable ranges up to $\pm 16g$ and $\pm 2000^\circ/s$, an on-chip FIFO and a wake-on-motion interrupt, which together make impact and tilt detection practical at low power.
- **Optional Redundant Impact Channel:** The ADXL345 three-axis accelerometer offers built-in tap, activity and free-fall interrupt engines on an independent bus address, allowing a second opinion on high-energy events at negligible cost.
- **Power and Protection:** A lithium-ion cell with a dedicated battery management system (BMS) — already available with the team — supplies the unit and enforces over-charge, over-discharge, over-current and short-circuit protection, so the logger survives supply interruption during a crash.

Verdict: All components are inexpensive, readily available in the Indian market, electrically compatible at 3.3 V logic, and require only moderate integration effort.

Software Feasibility:

- **Development Tools:** ESP-IDF and the Arduino core for the ESP32-S3, both mature and fully documented for this part.
- **Mobile Application:** Flutter for Android, with well-maintained BLE and mapping plugins.
- **On-device Storage:** SQLite within the mobile application for ride history, and non-volatile flash on the microcontroller for buffered records.

- **Analysis:** Python with NumPy and Matplotlib for offline log analysis and threshold tuning.

Verdict: Every tool in the chain is open-source or free for academic use, keeping development cost effectively zero.

System Integration: Sensor data flows from the GNSS receiver and the inertial measurement unit into the ESP32-S3, where it is filtered, fused into ride statistics and evaluated against the crash rule. Results are published as BLE GATT characteristics and consumed by the Flutter application, which persists them to SQLite and raises a cancellable crash alert. No component of the critical path leaves the vehicle or the paired handset.

Verdict: Integration is technically achievable with the team's existing skill set, and the modular design leaves room for later extension.

(ii) ECONOMIC FEASIBILITY

Prototype Development Cost:

- ESP32-S3 development board: Rs. 450–700
- ATGM336H-5N GNSS module with active antenna: Rs. 250–400
- MPU6500 inertial measurement unit module: Rs. 120–220
- ADXL345 accelerometer module (optional secondary channel): Rs. 120–200
- Lithium-ion cell with BMS: *already available with the team* (indicative market value Rs. 300–500)
- Enclosure, mounting hardware, wiring harness and connectors: Rs. 250–450
- Miscellaneous (perfboard or PCB, headers, passives, fasteners): Rs. 200–350

Total Incremental Hardware Cost: Rs. 1,270–2,120 per unit (excluding the battery and BMS already in hand).

Total Build Cost if Purchased Outright: Rs. 1,570–2,620 per unit.

Software Cost: Nil. ESP-IDF, the Arduino core, Flutter, SQLite and Python are all free; the Google Maps API remains within its free usage tier at the scale of a single prototype.

- No SIM card, no cellular data plan and no recurring subscription of any kind.
- No cloud hosting or database charges, since all storage is on the device and the paired handset.

Cost-Benefit Perspective: Commercial two-wheeler telematics devices in India typically cost several thousand rupees and additionally require a monthly data plan. SmartRide removes the recurring cost entirely and keeps the one-time cost within the price of a routine service. Against this, the rider gains a permanent ride record, automatic crash alerting and maintenance reminders tied to actual distance travelled. The low unit cost and the absence of any subscription make the design attractive both as an academic prototype and as a candidate for mass-market retrofit.

Contents

1	INTRODUCTION	1
1.1	BACKGROUND AND CONTEXT	1
1.2	PROBLEM STATEMENT	2
1.3	OBJECTIVES	3
1.4	SCOPE AND LIMITATIONS	4
1.4.1	Scope	4
1.4.2	Limitations	5
1.5	INDUSTRY RELEVANCE	5
2	LITERATURE REVIEW	7
2.1	SUMMARY OF EXISTING WORK	7
2.2	GAPS IDENTIFIED	9
2.3	CONTRIBUTION OF THE PROPOSED WORK	11
3	METHODOLOGY	14
3.1	APPROACH	14
3.2	THEORETICAL FRAMEWORK	15
3.2.1	Positioning, Speed and Distance	16
3.2.2	Impact and Orientation Mechanics	16
3.2.3	Communication Model	17
3.3	TOOLS AND TECHNOLOGIES	18
3.3.1	Hardware Components	18
3.3.2	Software Tools	19
3.4	DESIGN PROCESS	20
3.4.1	Use Case Diagram for the Ride Logging and Crash Detection System	20
3.4.2	System Overview: SmartRide Data Flow (Level 0 DFD)	21
3.4.3	System Data Management and Delivery (Level 1 DFD)	23
3.4.4	Data Processing Flow within the ESP32-S3 Micro- controller	25
3.4.5	Sequence Diagram	27

3.5	WORK PLAN	28
4	REFERENCES	29

List of Tables

0.1	Mapping of Project with Program Outcomes (POs)	VI
0.2	Mapping of Project with Course Outcomes (COs)	VII
0.3	Mapping of Project with Sustainable Development Goals (SDGs) .	VIII
2.1	Summary of Existing Work and Identified Gaps	9

List of Figures

3.1	System Architecture of SmartRide	15
3.2	Use Case Diagram of the SmartRide System	20
3.3	Level 0 DFD: SmartRide Context Diagram	21
3.4	Level 1 DFD: SmartRide Data Management and Delivery	23
3.5	Data Processing Flow within the ESP32-S3 Microcontroller	25
3.6	Sequence Diagram of the SmartRide Ride Logging and Crash De- tection System	27
3.7	Initial Work Plan of the SmartRide Project	28

ABBREVIATIONS

ADC	Analog-to-Digital Converter
ADXL345	Analog Devices Three-Axis Digital Accelerometer, Model 345
API	Application Programming Interface
ATGM336H	Zhongke Micro Multi-Constellation GNSS Module Series
BDS	BeiDou Navigation Satellite System
BLE	Bluetooth Low Energy
BMS	Battery Management System
CEP	Circular Error Probable
DFD	Data Flow Diagram
DMP	Digital Motion Processor
ESP32-S3	Espressif 32-bit Xtensa LX7 Dual-Core SoC with Wi-Fi and BLE
ESP-IDF	Espressif IoT Development Framework
FIFO	First In, First Out (sensor sample buffer)
GATT	Generic Attribute Profile
GLONASS	Global Navigation Satellite System (Russia)
GNSS	Global Navigation Satellite System
GPIO	General Purpose Input/Output
GPS	Global Positioning System
GSM	Global System for Mobile Communications
IDE	Integrated Development Environment
IMU	Inertial Measurement Unit
IoT	Internet of Things
I ² C	Inter-Integrated Circuit (serial bus)
MPU6500	InvenSense Six-Axis Motion Tracking Device, Model 6500
NavIC	Navigation with Indian Constellation
NMEA	National Marine Electronics Association (sentence format)
NVS	Non-Volatile Storage
PPS	Pulse Per Second
PSRAM	Pseudo-Static Random Access Memory
RTOS	Real-Time Operating System
SDG	Sustainable Development Goal
SoC	System on Chip
SPI	Serial Peripheral Interface
SQLite	Embedded Relational Database Engine
UART	Universal Asynchronous Receiver/Transmitter
UUID	Universally Unique Identifier

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND AND CONTEXT

Two-wheelers are the dominant mode of personal transport in India. They account for the largest share of registered vehicles in the country, and in a state such as Kerala they are the everyday vehicle of students, daily commuters, delivery riders and small traders alike. Their affordability, fuel economy and ability to negotiate congested roads make them indispensable. The same characteristics, however, leave the rider far more exposed than the occupant of a car: there is no crumple zone, no seat belt and no airbag, and a fall at even moderate speed can be serious.

Despite this exposure, the two-wheeler has remained almost entirely undigitised. A modern car routinely records trip data, warns about servicing, and in many markets places an automatic emergency call after a collision. A typical motorcycle or scooter offers none of this. The odometer shows a single cumulative number, the trip meter is reset by hand, and the rider has no record of where a ride went, how fast it was, how long it took or how harshly the vehicle was handled. Servicing intervals are recalled from memory or estimated from the last workshop visit rather than derived from distance actually travelled.

The safety gap is more consequential still. When a rider falls on a quiet stretch of road — and a large share of two-wheeler crashes in India occur away from dense traffic — there may be nobody present to raise an alarm. The interval between the impact and the moment somebody is informed is the single most important determinant of the outcome of a serious injury, and it is precisely this interval that no low-cost consumer product currently addresses for two-wheelers.

Commercial telematics solutions do exist, but their design assumptions make them a poor fit for this problem. They are built around a cellular modem, which brings with it the cost of the module itself, a SIM card, a recurring data plan and dependence on network coverage that is unreliable on exactly the rural roads where help is most needed. They also route personal movement data through a vendor cloud, which raises both privacy questions and the prospect of the device becoming useless if the service is discontinued. The result is a category of product priced and provisioned for fleet operators rather than for the individual rider.

At the same time, the components required to solve the problem locally have become inexpensive and capable. Modern microcontrollers such as the Espressif ESP32-S3 combine a dual-core processor, generous memory and a Bluetooth 5

Low Energy radio in a single low-cost module. Multi-constellation satellite receivers such as the ATGM336H-5N deliver usable position and speed fixes for a few hundred rupees. Six-axis inertial measurement units such as the MPU6500 provide acceleration and angular-rate data at rates and ranges that are entirely adequate for recognising an impact and a fall. Lithium-ion cells with integrated battery management have become a standard, safe and readily available building block. Meanwhile, every rider already carries a capable computer with a colour display, a map application and persistent storage — a smartphone.

SmartRide is built on the observation that these facts, taken together, remove the need for the cellular link altogether. If the on-board unit performs its own sensing, filtering, ride computation and crash reasoning, and if the smartphone in the rider's pocket serves as the display and the archive, then the only communication link required is a short Bluetooth Low Energy connection between the two. There is no SIM card, no data plan, no cloud account and no dependence on network coverage. The system works as well on an unlit rural road as it does in the middle of a city, and the rider's movement history never leaves the two devices they own.

1.2 PROBLEM STATEMENT

Riders of two-wheelers in India have no affordable, self-contained means of recording how their vehicle is ridden and of summoning help automatically when they fall.

The absence of a ride record deprives the rider of speed, distance, duration and route history for any journey, and therefore of any objective basis for reviewing their own riding or substantiating it to an insurer or employer. The absence of automatic crash notification means that a fall on an isolated road can remain unnoticed until somebody happens to pass, so that help is delayed at precisely the point where delay matters most. The absence of distance-linked maintenance tracking means that servicing is scheduled by guesswork rather than by actual kilometres covered, which shortens the life of the vehicle and increases both its running cost and its emissions.

Existing telematics products do not close this gap for the individual rider. They are expensive, they require a SIM card and a recurring data plan, they depend on cellular coverage that fails on rural roads, and they lock the user's movement data into a proprietary cloud service. Conventional academic accident-alert prototypes based on GSM or GPRS inherit the same connectivity dependence, and additionally tend to trigger on any large acceleration, producing false alarms from potholes, speed breakers and kerb strikes that quickly destroy the rider's trust in

the device.

There is therefore a need for a low-cost, self-contained on-board unit that logs ride data, recognises genuine crashes from motion signatures with an acceptably low false-alarm rate, and delivers this information to the rider's smartphone over Bluetooth Low Energy, without relying on cellular connectivity or subscription services.

1.3 OBJECTIVES

The objectives of this project are as follows:

- **To design a compact ESP32-S3 based on-board unit powered from a lithium-ion cell with an integrated battery management system.** The ESP32-S3 was selected for its dual-core Xtensa LX7 processor, its generous on-chip memory and its native Bluetooth 5 Low Energy radio, which together allow sensor acquisition, ride computation and BLE service hosting to run concurrently on one inexpensive module. Powering the unit from its own managed cell rather than directly from the vehicle supply keeps the logger alive through the electrical disturbance of a crash.
- **To acquire location, speed, course and distance from an ATGM336H-5N multi-constellation GNSS receiver.** The module delivers standard NMEA sentences over UART together with a one-pulse-per-second timing output, from which the firmware derives instantaneous speed, cumulative distance by the Haversine formula, and the travelled route as a sequence of timestamped coordinates.
- **To detect impacts and falls from MPU6500 accelerometer and gyroscope data.** The six-axis inertial measurement unit is sampled at a fixed high rate through its on-chip FIFO, and the firmware evaluates resultant acceleration together with orientation derived from the gyroscope in order to distinguish a genuine fall from ordinary road disturbance.
- **To provide an optional secondary impact channel using an ADXL345 accelerometer.** Where additional confidence is required, the ADXL345 can be populated on the same bus at a separate address and configured to raise a hardware interrupt through its built-in tap and activity engines, giving an independent corroboration of a high-energy event without occupying processor time.
- **To implement multi-condition crash detection that suppresses false**

alarms. Rather than triggering on acceleration alone, the firmware requires an impact, a sustained tilt and a period of post-impact stillness to occur in sequence before a crash is declared, so that potholes, speed breakers and kerb strikes are rejected.

- **To stream ride data and crash events to an Android application over Bluetooth Low Energy.** The on-board unit acts as a BLE GATT server exposing ride statistics, route points and crash notifications as characteristics, and continues to log and buffer data locally when the phone is out of range.
- **To store ride history on the handset and generate distance-based maintenance reminders.** The Flutter application persists each completed ride in a local SQLite database and maintains a cumulative odometer from which servicing reminders are raised at user-configurable intervals.
- **To keep the entire system independent of cellular networks and cloud services.** Every element of the critical path — sensing, computation, crash reasoning, alerting and storage — resides either on the on-board unit or on the paired handset, so that the system requires no SIM card, no data plan and no vendor account.
- **To validate accuracy, latency and reliability through controlled and on-road trials.** Distance and speed are to be compared against GNSS ground truth, crash detection is to be characterised for detection latency and false-alarm rate, and the BLE link is to be verified over typical rider-to-phone separation.

1.4 SCOPE AND LIMITATIONS

1.4.1 Scope

- The system targets a single two-wheeler paired with a single Android smartphone.
- Fully offline operation: ride data and crash events are stored on the on-board unit and on the handset, with no cloud or cellular component in the critical path.
- Ride parameters covered are instantaneous speed, cumulative distance, ride duration, travelled route and the timestamp of each record.
- Crash detection from inertial data, with the resulting alert raised inside the mobile application and cancellable by the rider within a countdown window.

- An Android application developed with Flutter, comprising a live dashboard, a route map, a searchable ride history backed by SQLite, and distance-based maintenance reminders.
- Optional population of the ADXL345 as a redundant impact channel, selectable at build time without any change to the application.

1.4.2 Limitations

- The unit does not place an emergency call or send an SMS to responders, because no cellular modem is fitted; the alert is delivered to the paired handset only. Onward notification to an emergency contact is identified as future work once a companion cellular variant is considered.
- Crash detection is threshold- and rule-based rather than learned. No machine-learning classifier is trained in the current phase, and the thresholds are calibrated empirically rather than derived from a large labelled crash dataset.
- Because the design is deliberately offline, a crash cannot be reported if the paired handset is absent, powered off or out of Bluetooth range at the moment of the event; the unit will, however, retain the event in its local buffer and deliver it on the next connection.
- Positioning accuracy is limited by the GNSS module and by sky visibility. Performance degrades in dense urban canyons, tunnels and heavy tree cover, and no dead-reckoning fallback is implemented in this phase.
- Only Android is supported. An iOS build, while feasible with the same Flutter codebase, is outside the present scope.
- The system does not interface with the vehicle’s electronic control unit, braking system or any factory electronics; it is a purely additive, non-intrusive retrofit.
- Validation is limited to the number of controlled drop, tilt and on-road trials that can be conducted safely by the team; no instrumented crash testing is performed.

1.5 INDUSTRY RELEVANCE

Connected-vehicle technology is one of the fastest-growing segments of the automotive industry, and two-wheelers are its largest unserved market. Telematics has matured rapidly in four-wheelers, where trip logging, usage-based insurance, predictive maintenance and automatic collision notification are becoming standard. Very little of that capability has reached the two-wheeler, largely because the cost

structure of conventional telematics — cellular module, SIM, data plan and cloud back end — cannot be supported at motorcycle price points.

SmartRide is relevant to this transformation precisely because it inverts that cost structure. By performing sensing, computation and crash reasoning at the edge and using the rider’s own smartphone as the display and archive, it demonstrates that the core value of telematics — a trustworthy ride record and automatic crash notification — can be delivered without any recurring cost. This edge-first, phone-as-gateway pattern is the same one now being adopted across wearables, asset trackers and industrial condition-monitoring products, and the architecture presented here is directly transferable to those domains.

Several industry segments map onto the work. The insurance sector is moving towards usage-based and behaviour-based premiums, for which an auditable record of distance, speed and harsh-event history is exactly the required input. Two-wheeler manufacturers and accessory suppliers in India are actively adding connected features to attract younger buyers, and a BLE-only module represents the least expensive route to that feature set. Fleet operators in the rapidly expanding food-delivery and last-mile logistics sector run large two-wheeler fleets on which crash detection and distance-accurate servicing have immediate operational value. Vehicle service networks benefit from maintenance scheduling driven by real distance rather than by calendar approximation.

The specific component choices also reflect current industry practice. The ESP32-S3 is widely deployed in commercial connected products; multi-constellation receivers of the ATGM336H class are standard in cost-sensitive tracking hardware; MEMS inertial measurement units of the MPU6500 family are used across consumer and automotive-adjacent applications; and lithium-ion packs with integrated battery management are the accepted approach to safe portable power. Building the prototype from this parts palette means the design is not a laboratory curiosity but a plausible product architecture that could be transitioned to a custom board with modest additional effort.

Finally, the project aligns with the national and international policy emphasis on road safety. Reducing deaths and injuries from road traffic accidents is an explicit target of the United Nations Sustainable Development Goals, and India’s own road-safety programmes identify two-wheeler riders as the most vulnerable category of road user. A retrofit device that shortens the time to help, and that costs less than a routine service, addresses that priority at a price point at which it can realistically be adopted at scale.

CHAPTER 2

LITERATURE REVIEW

2.1 SUMMARY OF EXISTING WORK

Research on two-wheeler safety and ride telematics has developed along three largely separate lines: satellite-based tracking of vehicle position, inertial detection of accidents, and rider-assistance systems built on additional sensing. Each line has produced capable results, but the literature contains relatively few attempts to combine all three in a single low-cost unit, and almost none that do so without assuming continuous network connectivity.

The tracking line of work is well established. A representative recent contribution is the development of real-time GPS tracking with estimated time of arrival on an Android application using an ESP32 and a u-blox NEO6MV2 receiver, presented at the IEEE International Conference on Computing, Communication and Networking Technologies in 2024. That work demonstrates convincingly that an inexpensive microcontroller paired with a consumer-grade satellite receiver can acquire position reliably, compute derived quantities such as arrival time, and deliver them to a mobile application with acceptable latency. It confirms the viability of the microcontroller-plus-GNSS pairing adopted in the present project. Its scope, however, stops at tracking: there is no inertial sensing, no accident detection and no maintenance logic, and the design assumes a network path to the phone rather than a direct local link.

A second line of work addresses rider safety through the Internet of Things more broadly. Chhabra et al., writing in *Scientific Reports* in 2024, propose an IoT-based smart framework for the safe driving experience of two-wheelers, integrating sensing and alerting into a connected architecture intended to improve rider outcomes. The contribution is valuable in framing rider safety as a systems problem rather than a single-sensor problem, and in demonstrating the benefit of coupling sensing to timely alerting. The framework nevertheless depends on continuous network connectivity for its alerting path, which means that its effectiveness degrades in exactly the low-coverage rural settings where a two-wheeler crash is most likely to go unwitnessed.

A third line pursues higher-capability rider protection through additional sensing. Alai, Rajamani and Sanjaya, in *Mechanical Systems and Signal Processing* in 2024, describe a smart e-scooter with embedded estimation of rear-vehicle trajectories for rider protection. The work is technically sophisticated, showing that

meaningful estimation of the surrounding traffic can be performed on an embedded two-wheeler platform. Its requirements, however, place it in a different cost class: ranging sensors and substantially greater computational capability are needed, which makes the approach difficult to justify for a mass-market retrofit unit intended to cost a small fraction of the vehicle’s service bill.

Alongside these, a large body of academic prototypes implements accident alerting using GSM or GPRS. The common pattern is an accelerometer that triggers above a fixed threshold, whereupon a microcontroller composes an SMS containing GPS coordinates and sends it to a stored contact number. These systems are simple, inexpensive in components and widely reproduced, and they establish the basic feasibility of automatic crash notification. Their weaknesses are equally well documented. They incur a recurring SIM cost; they fail where cellular coverage is poor, which is disproportionately where help is most needed; and because they trigger on a single acceleration threshold they are prone to false alarms from potholes, speed breakers, kerb strikes and even the vehicle being knocked over while parked. Repeated false alerts erode the user’s confidence and, in practice, lead to the device being disabled.

Two further strands of the literature inform the design choices made here. Work on Bluetooth Low Energy for sensor-to-phone communication has established the GATT model as an efficient, low-power means of exposing structured sensor data to a handset, with power consumption low enough for battery-operated devices and range adequate for a device carried on the same vehicle as its user. Separately, work on inertial fall and impact detection — much of it originating in wearable and elderly-care research — has shown that combining multiple conditions, typically an acceleration peak followed by an orientation change and a period of inactivity, markedly reduces false positives compared with any single threshold. This multi-condition principle, developed largely outside the vehicular context, is directly applicable to two-wheeler crash detection and is adopted in the present work.

The table below summarises the principal works reviewed, their contributions and the gaps they leave.

Work	Contribution	Gap / Takeaway
Real-Time GPS Tracking and ETA on an Android Application using ESP32 and u-blox NEO6MV2 (IEEE ICCCNT, 2024)	Demonstrates ESP32 with a consumer GNSS receiver delivering position and estimated arrival time to an Android application.	Tracking only; no crash detection and no maintenance logic. Confirms the microcontroller-plus-GNSS pairing adopted here.
Chhabra et al., <i>IoT Based Smart Framework for the Safe Driving Experience of Two Wheelers</i> , Scientific Reports (2024)	IoT framework addressing rider safety through coupled sensing and alerting.	Depends on continuous network connectivity; the present design keeps the critical path fully offline over BLE.
Alai et al., <i>A Smart E-Scooter with Embedded Estimation of Rear Vehicle Trajectories for Rider Protection</i> , MSSP (2024)	Embedded estimation of surrounding traffic on a two-wheeler platform.	Requires ranging sensors and higher compute; too costly for a mass-market retrofit unit.
Conventional GSM/GPRS accident-alert systems (widely reported academic prototypes)	Accelerometer-triggered SMS alerts carrying GPS coordinates to a fixed contact.	Recurring SIM cost, poor rural coverage and high false-alarm rates; motivates the multi-condition crash logic used here.
BLE GATT sensor-to-phone literature	Establishes GATT as a low-power, structured means of exposing sensor data to a handset.	Validates BLE as the sole communication link, with the phone as client and the unit as server.
Inertial fall-detection literature (wearable and elderly-care domain)	Shows that combining impact, orientation change and post-event inactivity greatly reduces false positives.	Principle transfers directly to two-wheeler crash detection and is adopted in this work.

Table 2.1: Summary of Existing Work and Identified Gaps

2.2 GAPS IDENTIFIED

A review of the work above reveals consistent limitations that, taken together, explain why no affordable two-wheeler ride-and-safety unit is in general use. The following gaps have been identified:

- **Mandatory dependence on cellular connectivity.** Almost every safety-oriented system reviewed assumes a GSM, GPRS or Wi-Fi path for its alerting function. This makes the device useless in low-coverage areas, imposes a recurring data cost on the user, and introduces a failure mode that is strongly correlated with the very locations where a crash is most likely to go unnoticed.
- **Single-threshold crash detection and consequent false alarms.** The prevailing approach declares an accident whenever acceleration crosses a fixed limit. Ordinary riding in Indian conditions routinely produces such peaks from pot-holes, speed breakers, kerb strikes and abrupt braking. The resulting false alerts are the single most frequently reported practical failure of these prototypes, and they lead directly to the device being switched off.
- **Separation of tracking, safety and maintenance functions.** Tracking systems log position but do not detect crashes; accident-alert systems detect impacts but keep no ride history; neither addresses servicing. The rider is therefore offered three partial products rather than one coherent one, and no single reviewed system integrates all three.
- **Absence of distance-based maintenance intelligence.** Although cumulative distance is trivially available once position is being logged, none of the reviewed systems uses it to drive servicing reminders. Maintenance consequently remains scheduled by guesswork, with measurable cost to vehicle life and emissions.
- **Cost and complexity of higher-capability approaches.** Systems that add ranging sensors or significant computation achieve impressive rider protection but at a price and power budget incompatible with a retrofit unit for the existing vehicle population.
- **Cloud lock-in and loss of data ownership.** Commercial telematics routes the rider's movement history through a proprietary back end. This raises privacy concerns over continuous location data and leaves the device dependent on the continued operation of a commercial service.
- **No graceful behaviour when the link is unavailable.** Most reviewed designs assume the communication channel is present. Few describe what happens to data recorded while the channel is down, so records generated during a coverage gap are simply lost.
- **Inadequate attention to power integrity during a crash.** Systems draw-

ing power directly from the vehicle supply are vulnerable to exactly the electrical disturbance a crash produces. Few reviewed designs discuss how the unit stays alive long enough to report the event it has just detected.

- **Limited rider control over the alert.** Where an alert is raised, it is often sent immediately and irrevocably. Without a cancellation window, a false positive propagates to the emergency contact, compounding the trust problem described above.
- **Sparse reporting of validation methodology.** Detection latency, false-alarm rate and distance accuracy are seldom characterised systematically, which makes it difficult to compare approaches or to establish that thresholds generalise beyond the authors’ own trials.

Taken together, these gaps point to the need for a low-cost, self-contained on-board unit that integrates ride logging, multi-condition crash detection and distance-based maintenance in one device, that keeps its critical path entirely offline, that buffers data when the link is down, that remains powered through an impact, and that leaves the rider in control of the alert.

2.3 CONTRIBUTION OF THE PROPOSED WORK

The proposed project, *SmartRide: Bluetooth-Based Two-Wheeler Ride Logger and Crash Detection System Using ESP32-S3*, addresses the identified gaps through the following contributions:

- **A fully offline, network-independent architecture.** SmartRide eliminates the cellular link entirely. Sensing, filtering, ride computation, crash reasoning and storage all take place on the on-board unit and the paired handset, connected by a short Bluetooth Low Energy link. The system therefore functions identically on a rural road and in a city, requires no SIM card or data plan, and incurs no recurring cost.
- **Multi-condition crash detection that suppresses false alarms.** Instead of a single acceleration threshold, the firmware requires three conditions in sequence: a resultant-acceleration peak above a calibrated limit, a sustained tilt beyond an orientation bound derived from gyroscope integration, and a period of near-stillness following the impact. Because a pothole produces an acceleration spike without a sustained tilt, and a low-speed manoeuvre produces tilt without an impact, the conjunction of all three rejects the dominant false-alarm sources reported in the literature.

- **An optional redundant impact channel.** The ADXL345 may be populated alongside the MPU6500 on the same bus at a distinct address. Its built-in tap and activity engines raise a hardware interrupt independently of the main sampling loop, providing corroboration of a high-energy event from a physically separate device. This addresses sensor-level single-point failure at negligible cost, and the firmware degrades gracefully to MPU6500-only operation when the part is not fitted.
- **Integration of ride logging, safety and maintenance in one unit.** A single device provides the ride record, the crash alert and the servicing reminder. Cumulative distance, already computed for the ride log, is reused to drive maintenance reminders at user-configurable intervals, so the maintenance feature is obtained at essentially no additional hardware cost.
- **Edge computation on the ESP32-S3 with the phone as a pure client.** All ride statistics are computed on the microcontroller and exposed as BLE GATT characteristics. The application reads and renders them but performs no essential computation, which means the unit keeps logging correctly whether or not the phone is connected, and the same unit would work unchanged against a future iOS client.
- **Local buffering with deferred delivery.** Records and crash events generated while the handset is absent or out of range are written to non-volatile storage on the unit and transferred on the next successful connection. No data is lost to a link outage, which directly addresses one of the gaps identified above.
- **Independent, crash-survivable power.** The unit runs from its own lithium-ion cell managed by a dedicated battery management system providing over-charge, over-discharge, over-current and short-circuit protection. Because the logger does not depend on the vehicle supply at the moment of impact, it remains alive to report the event it has just detected.
- **Multi-constellation positioning suited to Indian conditions.** The ATGM336H-5N receives from several satellite constellations concurrently, which improves fix availability and the time to first fix relative to a single-constellation receiver under obstructed sky. Its one-pulse-per-second output is used to discipline record timestamps so that logged data remains temporally consistent.
- **A rider-cancellable alert.** A crash event raises a countdown in the application which the rider can dismiss. This keeps the human in the loop, ensures that a

residual false positive does not propagate, and preserves the rider’s trust in the device — the practical precondition for it remaining switched on.

- **Privacy by architecture.** Because no movement data leaves the vehicle and the rider’s own handset, location privacy is a structural property of the design rather than a policy promise, and the device cannot be rendered inoperable by the discontinuation of a commercial service.
- **A defined validation methodology.** Distance and speed accuracy, crash detection latency, false-alarm behaviour under deliberately adverse road inputs, and BLE link reliability over realistic rider-to-phone separation are each to be characterised explicitly, addressing the weak validation reporting observed in the reviewed literature.

CHAPTER 3

METHODOLOGY

3.1 APPROACH

The proposed system, *SmartRide*, follows a hybrid experimental and analytical methodology in which hardware bring-up, firmware development and mobile application development proceed in parallel and are brought together through incremental integration. The guiding principle is that every function on the critical path must execute either on the on-board unit or on the paired handset, so that the system never depends on a network to do its job.

Work begins with sensor bring-up on a breadboard. The ATGM336H-5N GNSS receiver is connected to a hardware UART of the ESP32-S3 and its NMEA output is parsed and validated against known coordinates; the MPU6500 inertial measurement unit is connected over its serial interface and its accelerometer and gyroscope scales are verified against static gravity and controlled rotation. Where the optional ADXL345 is populated, it is placed on the same I²C bus at its alternate address and its interrupt output is routed to a free GPIO. This stage establishes, for each device, a known-good register configuration, a sampling rate and a noise characterisation that the later stages depend upon.

The firmware is then developed as a set of cooperating real-time tasks rather than a single polling loop. A high-rate task services the inertial measurement unit through its on-chip FIFO so that no samples are lost to scheduling jitter; a low-rate task parses GNSS sentences as they arrive; a computation task consumes both streams from queues and derives ride statistics; and a communication task maintains the Bluetooth Low Energy connection and publishes updated characteristic values. Separating these concerns means that a slow or absent phone connection cannot delay sensor acquisition, and that the crash rule is evaluated on a deterministic schedule.

Ride statistics are computed on the microcontroller itself. Instantaneous speed is taken from the GNSS velocity solution rather than differentiated from position, which is substantially less noisy. Incremental distance between consecutive accepted fixes is computed with the Haversine formula and accumulated; fixes are accepted only when the receiver reports a valid solution and the implied speed is physically plausible, which prevents the cumulative odometer from being corrupted by outliers. Ride duration is taken from timestamps disciplined by the receiver's one-pulse-per-second output so that records remain temporally consistent across

a long ride.

Crash detection is treated as a sequential decision rather than a single comparison. The firmware first looks for an impact in the resultant acceleration, then requires the vehicle’s orientation to have changed and remained changed, and finally requires a period of near-stillness. Only when all three conditions have occurred in that order within a bounded time window is a crash declared. The design intent is explicitly to accept a slightly longer detection latency in exchange for a false-alarm rate low enough that the rider leaves the device switched on.

The mobile application is developed against the unit’s BLE GATT service using Flutter. It subscribes to the ride and crash characteristics, renders a live dashboard, plots the route on a map, and writes each completed ride into a local SQLite database from which history and the cumulative odometer are served. A crash notification raises a countdown that the rider can cancel, keeping the human in the loop.

Development is iterative. Each stage is validated before the next is layered on: sensor readings against physical ground truth, distance against known routes, the crash rule against controlled drop and tilt trials and against deliberately adverse road inputs such as speed breakers and kerb strikes, and the BLE link against realistic rider-to-phone separation. This closed-loop cycle — sense, compute, decide, deliver, measure, retune — is what converts a set of working components into a system whose behaviour can be trusted.

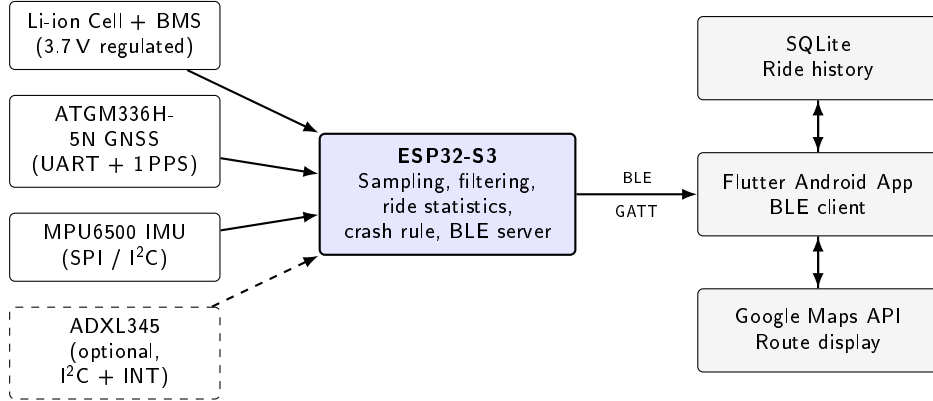


Figure 3.1: System Architecture of SmartRide

3.2 THEORETICAL FRAMEWORK

The theoretical foundation of SmartRide rests on three bodies of principle: the geodesy used to convert satellite fixes into distance travelled, the mechanics used to recognise an impact and a fall from inertial data, and the communication model used to expose the results to a handset.

3.2.1 Positioning, Speed and Distance

The GNSS receiver reports position as a latitude and longitude pair in the WGS-84 datum. The great-circle distance between two consecutive fixes (φ_1, λ_1) and (φ_2, λ_2) is obtained from the Haversine formula:

$$a = \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos\varphi_1 \cos\varphi_2 \sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (3.1)$$

$$d = 2R \operatorname{atan2}(\sqrt{a}, \sqrt{1-a}) \quad (3.2)$$

where $\Delta\varphi = \varphi_2 - \varphi_1$, $\Delta\lambda = \lambda_2 - \lambda_1$, and R is the mean Earth radius, taken as 6371 km. The formula is preferred over the simpler equirectangular approximation because it remains accurate for the short baselines between successive fixes without special handling near the poles, and it is inexpensive enough to evaluate on the microcontroller at the receiver's update rate.

The cumulative ride distance is the sum of accepted increments:

$$D = \sum_{i=1}^N d_i, \quad d_i \text{ accepted iff fix valid} \wedge \frac{d_i}{\Delta t_i} \leq v_{\max} \quad (3.3)$$

The plausibility gate $d_i/\Delta t_i \leq v_{\max}$ is essential in practice: a single spurious fix would otherwise inject a large spurious increment into the odometer that could never be removed. Instantaneous speed is taken directly from the receiver's velocity solution, which is derived from Doppler measurements and is markedly less noisy than a position difference:

$$v = v_{\text{GNSS}}, \quad \bar{v} = \frac{D}{T}, \quad T = \sum_i \Delta t_i \quad (3.4)$$

3.2.2 Impact and Orientation Mechanics

The MPU6500 reports acceleration along three orthogonal body axes. The resultant, or magnitude, of the acceleration vector is

$$a_r = \sqrt{a_x^2 + a_y^2 + a_z^2} \quad (3.5)$$

At rest this quantity equals $1g$ regardless of the orientation of the unit, which makes it a convenient orientation-independent impact indicator. An impact is flagged when a_r exceeds a calibrated threshold a_{th} , nominally in the region of $4g$ and tuned empirically during field trials.

Orientation is obtained by integrating the gyroscope's angular rates and correcting the result against the accelerometer's estimate of the gravity direction,

which bounds the drift inherent in integration alone. The tilt of the unit from the vertical follows from the accelerometer directly:

$$\theta = \arccos\left(\frac{a_z}{a_r}\right) \quad (3.6)$$

A fall is characterised not by a momentary tilt — a motorcycle leans substantially in normal cornering — but by a tilt that exceeds a bound and is then *sustained*:

$$\theta(t) > \theta_{\text{th}} \quad \text{for all } t \in [t_0, t_0 + \tau_{\text{tilt}}] \quad (3.7)$$

with θ_{th} nominally 60° . The final condition is stillness: over a short window following the impact, the variation of the resultant acceleration must remain small,

$$\max_{t \in [t_1, t_1 + \tau_{\text{still}}]} |a_r(t) - 1g| < \epsilon_{\text{still}} \quad (3.8)$$

The crash decision is the conjunction of the three, evaluated in order within a bounded window:

$$\text{Crash} = (a_r > a_{\text{th}}) \wedge (\theta \text{ sustained} > \theta_{\text{th}}) \wedge (\text{stillness}) \quad (3.9)$$

The value of the conjunction lies in what each term excludes. A pothole or speed breaker produces a large a_r but no sustained tilt, because the vehicle remains upright and continues moving. A slow-speed lean or a tight turn produces tilt without an impact. A vehicle stopped at a signal satisfies stillness alone. Only a genuine fall produces all three in sequence, which is why the multi-condition rule is markedly more selective than any single threshold.

3.2.3 Communication Model

Bluetooth Low Energy structures data exchange through the Generic Attribute Profile (GATT). The on-board unit acts as a GATT *server*, exposing a custom primary service whose characteristics carry ride statistics, route points and crash events; the handset acts purely as a *client*, subscribing to notifications and reading values on demand. This assignment of roles is deliberate. Because the unit is the server and the source of truth, it continues to sample sensors, compute statistics and evaluate the crash rule whether or not a client is connected, and records generated while the phone is absent are written to non-volatile storage and delivered on the next connection. The phone performs no essential computation, so the same firmware would serve an iOS or desktop client without modification.

3.3 TOOLS AND TECHNOLOGIES

3.3.1 Hardware Components

- **ESP32-S3 Microcontroller.** A dual-core Xtensa LX7 system on chip running at up to 240 MHz, with integrated 2.4GHz Wi-Fi and Bluetooth 5 Low Energy, 512kB of on-chip SRAM, optional external PSRAM and a rich complement of serial peripherals. It acts as the sole processor of the on-board unit: it reads the GNSS receiver over UART and the inertial measurement unit over SPI or I²C, computes ride statistics, evaluates the crash rule, maintains the non-volatile record buffer and hosts the BLE GATT service. The part was chosen over the original ESP32 for its greater processing headroom, its native USB interface, which simplifies flashing and log capture, and its Bluetooth 5 Low Energy radio.
- **ATGM336H-5N GNSS Receiver.** A compact multi-constellation satellite receiver that tracks several systems concurrently and outputs standard NMEA 0183 sentences over UART, together with a one-pulse-per-second timing signal. Concurrent multi-constellation reception improves fix availability and shortens time-to-first-fix under obstructed sky compared with a single-constellation receiver, which matters on tree-lined and built-up Kerala roads. The module supplies latitude, longitude, ground speed, course and satellite count, and its 1PPS output is used to discipline record timestamps.
- **MPU6500 Inertial Measurement Unit.** A six-axis device combining a three-axis accelerometer and a three-axis gyroscope with on-chip sixteen-bit conversion, selectable full-scale ranges up to $\pm 16g$ and $\pm 2000^\circ/\text{s}$, a configurable digital low-pass filter, an internal FIFO and a wake-on-motion interrupt. It supplies the acceleration and angular-rate streams from which impact and sustained tilt are derived. Reading through the FIFO rather than polling individual registers ensures that no samples are lost to scheduling jitter in the firmware.
- **ADXL345 Accelerometer (Optional).** A three-axis digital accelerometer with selectable ranges up to $\pm 16g$, thirteen-bit resolution and hardware detection engines for single tap, double tap, activity, inactivity and free fall, any of which can raise an interrupt on a dedicated pin. When populated, it sits on the same I²C bus at its alternate address and provides an independent, physically separate corroboration of a high-energy event without consuming processor time. The firmware treats its presence as optional and degrades gracefully to MPU6500-only operation when it is absent.

- **Lithium-Ion Cell with Battery Management System.** A single Li-ion cell, already available with the team, supplies the unit through a dedicated battery management system providing over-charge, over-discharge, over-current and short-circuit protection, with a regulator presenting a clean 3.3 V rail to the logic. Running the unit from its own managed cell rather than directly from the vehicle supply is a deliberate design decision: the logger must survive the electrical disturbance of the very event it is designed to report.
- **Enclosure, Mounting and Wiring.** A closed enclosure with vibration-isolating mounting protects the assembly and, importantly, fixes the orientation of the inertial measurement unit relative to the vehicle, which the tilt calculation depends upon. The GNSS antenna is positioned for clear sky visibility.

3.3.2 Software Tools

- **ESP-IDF and the Arduino Core for ESP32-S3.** The primary firmware development environment, providing the FreeRTOS kernel, peripheral drivers, the BLE host stack and the non-volatile storage subsystem used for the record buffer.
- **Flutter (Dart).** Used to build the Android application, giving a single code-base for the dashboard, map view, ride history and reminder logic, with mature plugins for Bluetooth Low Energy and mapping.
- **SQLite.** The embedded relational database within the application, holding completed rides, route point sequences, crash events and the cumulative odometer from which maintenance reminders are derived.
- **Google Maps API.** Used within the application to render the travelled route from the logged coordinate sequence.
- **Python with NumPy and Matplotlib.** Used offline to analyse captured sensor logs, to visualise impact and tilt signatures from controlled trials, and to tune the crash thresholds before they are committed to firmware.
- **Git and GitHub.** Version control for firmware, application and documentation, with the repository also serving as the record of design decisions across the phases.
- **Programming Languages.** C and C++ for the firmware, Dart for the mobile application, and Python for analysis.

3.4 DESIGN PROCESS

The design is presented through a use case diagram that fixes the boundary of the system and the interactions across it, a hierarchy of data flow diagrams that decompose the system first into its major components and then into the processing performed within the microcontroller, and a sequence diagram that establishes the temporal ordering of a ride and of a crash event. Together these views describe a system involving a Rider, an on-board unit comprising the ESP32-S3 with the ATGM336H-5N GNSS receiver and the MPU6500 inertial measurement unit, and a Flutter mobile application.

3.4.1 Use Case Diagram for the Ride Logging and Crash Detection System

The interactions fall into three groups: ride operation, safety, and history and maintenance.

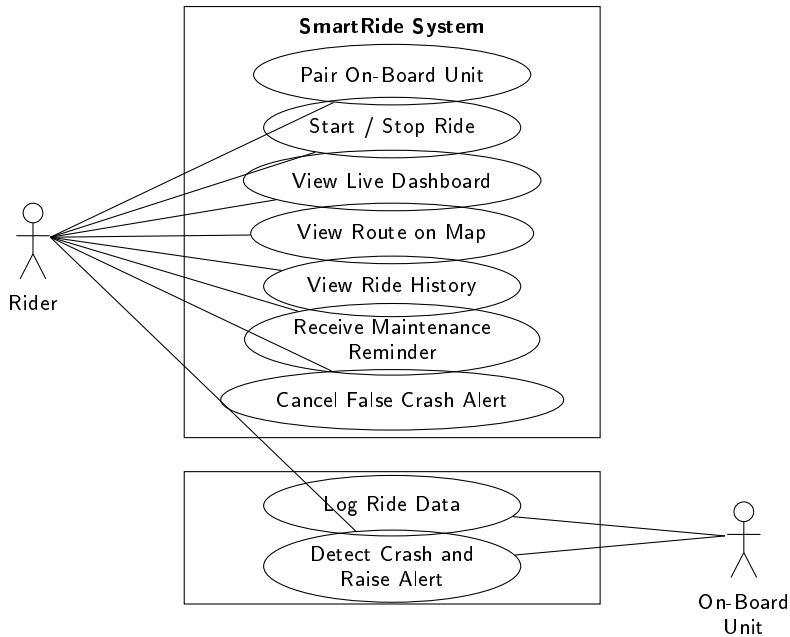


Figure 3.2: Use Case Diagram of the SmartRide System

(i) **Ride Operation.** The rider pairs the on-board unit with the handset once, after which the application reconnects automatically whenever the unit is in range.

- **Pair On-Board Unit:** a one-time Bluetooth Low Energy pairing that binds the unit to the rider's handset.
- **Start / Stop Ride:** delimits a ride so that statistics and route points are grouped into a single record. The unit also infers ride boundaries from sustained motion and stillness, so an explicit action is not required.

- **View Live Dashboard:** presents instantaneous speed, elapsed distance, ride duration and satellite status as the ride proceeds.
 - **View Route on Map:** plots the logged coordinate sequence over a map view.
- (ii) **Safety.** The safety interactions are the ones the rider hopes never to use.
- **Detect Crash and Raise Alert:** performed by the on-board unit, which evaluates the multi-condition rule continuously and notifies the application when all conditions are met.
 - **Cancel False Crash Alert:** the rider dismisses the countdown raised by the application, keeping a residual false positive from propagating further.
- (iii) **History and Maintenance.**
- **Log Ride Data:** performed by the on-board unit, which writes records to its non-volatile buffer and transfers them on connection.
 - **View Ride History:** browses past rides held in the handset's SQLite database.
 - **Receive Maintenance Reminder:** raised by the application when the cumulative odometer crosses a user-configured servicing interval.

3.4.2 System Overview: SmartRide Data Flow (Level 0 DFD)

The context diagram identifies what lies outside the system and what crosses its boundary. Three external entities are relevant. The **GNSS constellation** supplies the satellite signals from which position, speed and course are derived. The **vehicle and road** constitute the physical environment whose motion the inertial sensors observe. The **rider** supplies configuration and control and consumes the system's output.

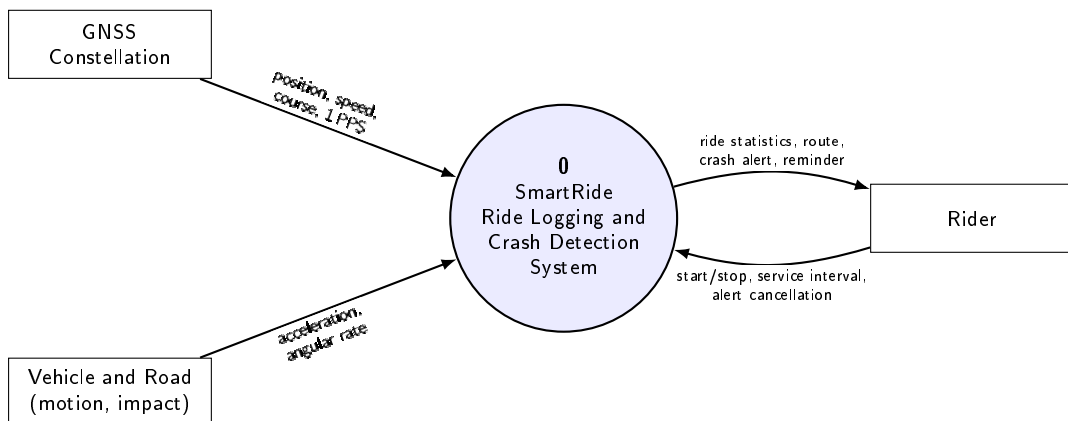


Figure 3.3: Level 0 DFD: SmartRide Context Diagram

The system comprises four core elements: the ATGM336H-5N GNSS receiver, the MPU6500 inertial measurement unit, the ESP32-S3 microcontroller and the

Flutter mobile application. Data flows from the sensing components through the microcontroller to the application, with a control path returning from the rider.

(i) ATGM336H-5N — GNSS Receiver. The module acquires and tracks signals from multiple satellite constellations concurrently and computes a navigation solution.

Process:

- It acquires satellite signals and resolves a position, velocity and time solution.
- It emits the solution as NMEA 0183 sentences over UART at a configured update rate.
- It asserts a one-pulse-per-second signal aligned to satellite time.

Output: latitude, longitude, ground speed, course, fix validity and satellite count, delivered to the ESP32-S3 over UART, with 1 PPS on a separate GPIO for timestamp discipline.

(ii) MPU6500 — Inertial Measurement Unit. The device measures linear acceleration and angular rate along three orthogonal axes.

Process:

- Micro-machined structures respond to acceleration and rotation, and the responses are digitised on chip at sixteen-bit resolution.
- A configurable digital low-pass filter removes out-of-band content before the samples reach the FIFO.
- Samples accumulate in the internal FIFO, from which the firmware reads them in blocks.

Output: time-ordered acceleration and angular-rate samples delivered to the ESP32-S3, from which the resultant acceleration and the tilt angle are computed. Where the ADXL345 is populated, it additionally asserts a hardware interrupt on a high-energy event.

(iii) ESP32-S3 — Microcontroller. The microcontroller is the entire intelligence of the on-board unit.

Process:

- **Acquisition:** it parses NMEA sentences as they arrive and drains the inertial FIFO on a deterministic schedule.
- **Computation:** it validates fixes, computes Haversine increments, accumulates distance and duration, and maintains the ride record.
- **Decision:** it evaluates the sequential crash rule over the inertial stream.

- **Persistence:** it writes records to non-volatile storage whenever no client is connected.
- **Publication:** it hosts the BLE GATT service and notifies subscribed clients of updated values and crash events.

Output: ride statistics, route points and crash events, published over Bluetooth Low Energy.

(iv) **Flutter Mobile Application.** The application is the presentation and archival layer.

Process: it connects as a BLE client, subscribes to the unit's characteristics, renders the dashboard and map, writes completed rides to SQLite, maintains the cumulative odometer and raises maintenance reminders and cancellable crash alerts.

Output: the rider's view of the system, and the durable ride archive.

3.4.3 System Data Management and Delivery (Level 1 DFD)

The Level 1 decomposition expands the single process of the context diagram into the processes that actually run, and names the two stores in which data persists.

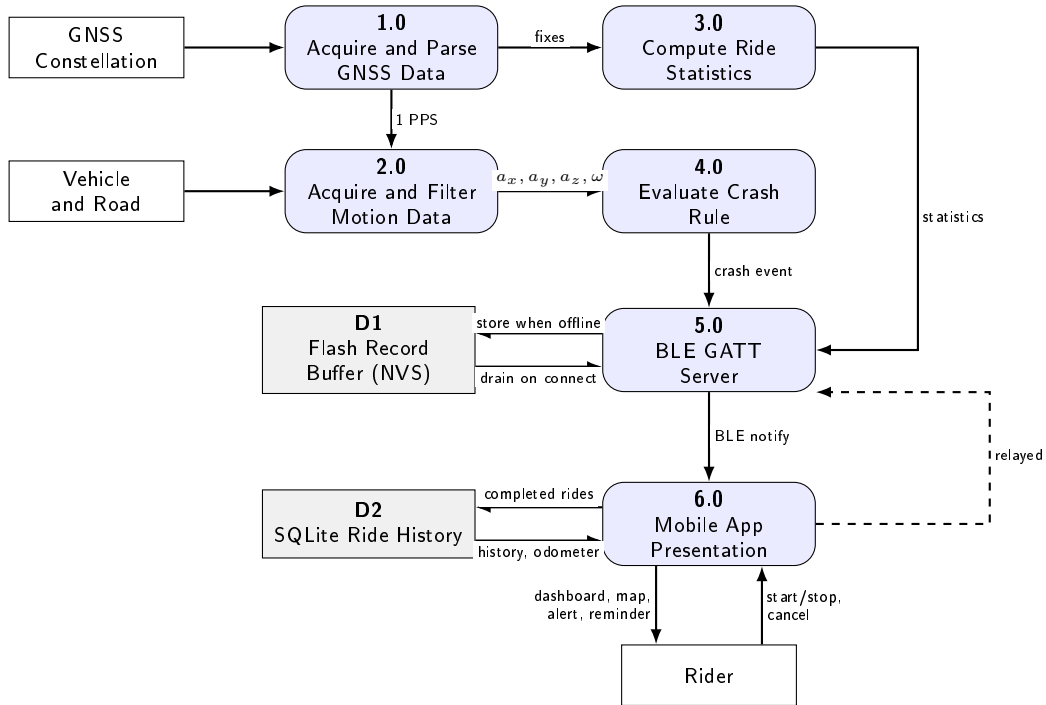


Figure 3.4: Level 1 DFD: SmartRide Data Management and Delivery

(i) Data Ingestion and Processing

Process 1.0 reads bytes from the UART attached to the GNSS receiver, re-assembles them into complete NMEA sentences, verifies each checksum and extracts latitude, longitude, ground speed, course, fix validity and satellite count.

Sentences failing the checksum are discarded silently; fixes marked invalid are not allowed to reach the distance computation.

Process 2.0 drains the inertial measurement unit’s FIFO in blocks, converts the raw sixteen-bit counts to physical units using the configured full-scale ranges, applies bias corrections established during calibration, and smooths the stream with a short moving average that removes sensor noise without masking the sharp transient of an impact. Where the optional ADXL345 is fitted, its interrupt line is monitored here as an independent high-energy indication.

Process 3.0 computes the ride statistics. It applies the plausibility gate to each candidate fix, evaluates the Haversine increment, accumulates distance and duration, and appends the coordinate to the route sequence. Process 4.0 evaluates the sequential crash rule — impact, sustained tilt, stillness — over the filtered inertial stream, maintaining the small amount of state required to recognise the conditions in order within a bounded window.

(ii) Persistence and Publication Paths

Output from the computation processes follows two paths. **Store D1**, a non-volatile flash region on the microcontroller, receives ride records whenever no client is connected, so that a ride taken without the handset present is not lost. **Process 5.0**, the BLE GATT server, publishes live statistics and crash events to a connected client, and on connection first drains any records held in D1 before resuming live notification. This ordering matters: the rider’s history remains complete and in sequence regardless of when the phone was in range.

(iii) Alert and User Delivery

Process 6.0, the mobile application, is the final interface. It writes completed rides and their route sequences into **Store D2**, the SQLite database on the handset, from which ride history is browsed and the cumulative odometer is maintained. It compares the odometer against the configured servicing interval and raises a maintenance reminder when the interval is crossed. On receiving a crash event it raises an alert with a countdown, which the rider may cancel; the cancellation is recorded and returned to the unit, both to stop the alert and to inform later threshold review.

In summary, the two sensing processes feed two computation processes, which branch into a persistence path through the flash buffer and a live publication path through the GATT server, converging at the mobile application where data is archived and presented to the rider.

3.4.4 Data Processing Flow within the ESP32-S3 Microcontroller

This diagram expands the processing internal to the microcontroller, showing how raw sensor input becomes a published record. The firmware is organised as a set of FreeRTOS tasks communicating through queues, so that acquisition, computation and communication proceed without blocking one another.

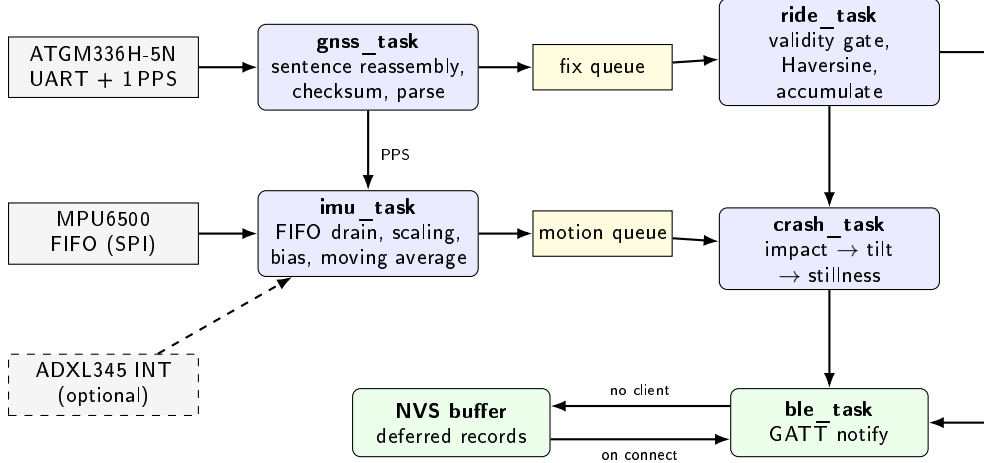


Figure 3.5: Data Processing Flow within the ESP32-S3 Microcontroller

(i) Data Input

Two independent input streams enter the microcontroller. The GNSS receiver delivers NMEA sentences as a byte stream on a hardware UART, typically at one update per second, together with a one-pulse-per-second edge on a GPIO. The inertial measurement unit accumulates samples in its FIFO at a substantially higher rate and is read in blocks over SPI. Where fitted, the ADXL345 contributes only an interrupt edge, consuming no bus bandwidth until an event occurs.

The disparity in rates is the reason the two streams are acquired by separate tasks. The inertial stream must be drained promptly to avoid FIFO overrun, whereas GNSS sentences arrive slowly but in bursts; handling both in a single loop would risk losing motion samples precisely during the transient that matters most.

(ii) Core Processing

1. **Sentence Parsing and Fix Validation.** The GNSS task reassembles sentences, verifies checksums and extracts the navigation fields. Fixes reported invalid, or implying an impossible speed relative to the previous accepted fix, are rejected before they can affect the odometer.
2. **Sample Conditioning.** The inertial task converts raw counts to physical units, subtracts the biases established during calibration and applies a short

moving average. The filter length is a deliberate compromise: long enough to suppress vibration-induced noise, short enough to preserve the rise time of a genuine impact.

3. **Ride Computation.** The ride task evaluates the Haversine increment between accepted fixes, accumulates distance and duration, tracks maximum and mean speed, and appends coordinates to the route sequence at a decimated rate appropriate to map display.
4. **Crash Evaluation.** The crash task maintains a small state machine. From the idle state, an impact above threshold moves it to a tilt-watch state; a tilt sustained beyond the orientation bound moves it to a stillness-watch state; stillness maintained for the required interval declares a crash. Failure of any condition within its window returns the machine to idle, which is the mechanism by which potholes and normal cornering are rejected.
5. **Record Formatting.** Results are packed into a compact binary record carrying timestamp, coordinates, speed, cumulative distance, and status flags, sized to fit within a single BLE notification so that no record requires fragmentation.

(iii) Communication and Output

1. **BLE Connection Management.** The BLE task advertises the custom service, accepts a connection from the bonded handset, and manages subscription state for each characteristic. Connection parameters are negotiated towards a longer interval once a ride is under way, trading latency for power, and towards a short interval when a crash event is pending.
2. **Publication and Deferred Delivery.** With a client connected, records are notified as they are produced. With no client connected, they are appended to the non-volatile buffer instead. On the next connection the buffer is drained in order before live notification resumes, so the handset receives a continuous history.
3. **Error Handling and Reliability.** The firmware monitors for FIFO overrun, UART framing errors, loss of satellite fix and BLE disconnection. Loss of fix suspends distance accumulation but does not suspend crash detection, since the inertial path is independent of positioning. Buffer writes are wear-levelled and record-framed, so that an unexpected power loss can corrupt at most the record being written and never the records already committed.

3.4.5 Sequence Diagram

The sequence proceeds in four phases: ride start, the logging loop, a crash event, and ride end.

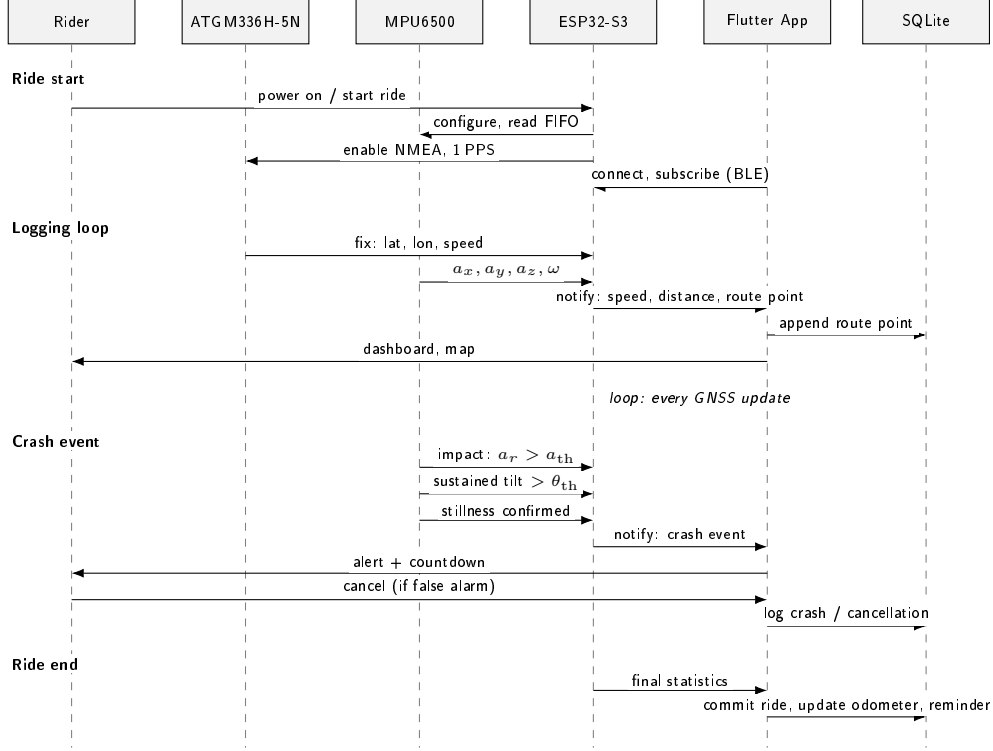


Figure 3.6: Sequence Diagram of the SmartRide Ride Logging and Crash Detection System

(i) **Ride Start Phase.** On power-up the ESP32-S3 configures both sensors — setting the inertial measurement unit’s ranges, filter bandwidth and FIFO, and enabling the required NMEA sentences and the 1 PPS output on the GNSS receiver — and begins advertising its BLE service. The handset, already bonded, connects and subscribes to the ride and crash characteristics. The unit begins logging immediately, whether or not the handset has connected.

(ii) **Logging Loop Phase.** For each GNSS update the receiver delivers a fix, which the microcontroller validates and converts into a distance increment and a route point. The inertial measurement unit meanwhile delivers motion samples at a much higher rate. The microcontroller notifies the current statistics to the application, which appends the route point to SQLite and refreshes the dashboard and map for the rider. This loop repeats for the duration of the ride.

(iii) **Crash Event Phase.** The three conditions of the crash rule arrive in sequence from the inertial stream: an impact above threshold, a tilt beyond the orientation bound sustained for the required interval, and a period of stillness.

Only on the third does the microcontroller emit a crash event. The application raises an alert with a countdown which the rider may cancel; the event and any cancellation are written to SQLite, both for the rider’s record and for subsequent threshold review.

(iv) Ride End Phase. On detecting sustained stillness, or on explicit instruction from the rider, the microcontroller emits the final ride statistics. The application commits the ride, updates the cumulative odometer and, if the servicing interval has been crossed, raises a maintenance reminder.

3.5 WORK PLAN

The preparation phase of the project runs from June to October 2026, covering topic finalisation and literature review through the economic feasibility study and component selection to the interim evaluation and the planning of the implementation phase. Each column in the chart below represents half a month.

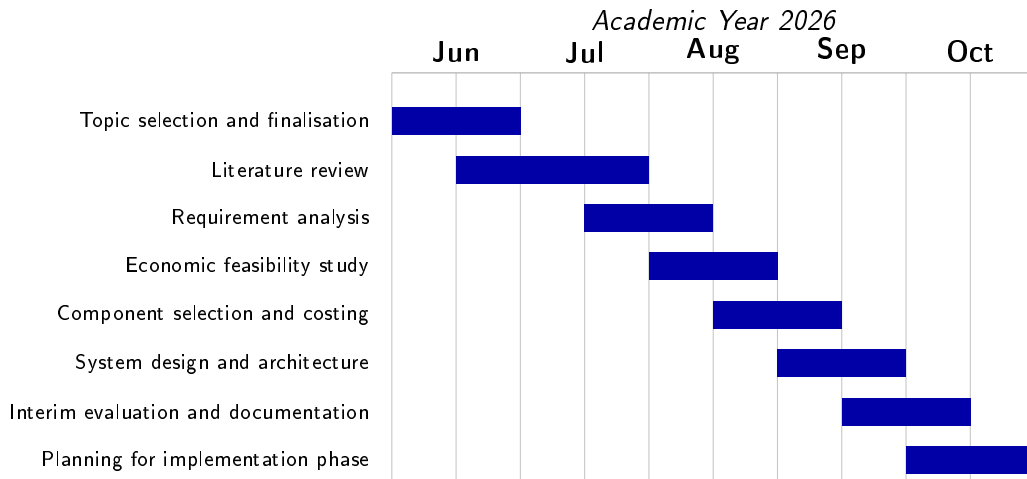


Figure 3.7: Initial Work Plan of the SmartRide Project

The chart reflects the actual progression of the preparation phase. Topic selection and literature review dominated the early months; requirement analysis and the economic feasibility study followed; component selection and costing in August produced the revised hardware set described in this report — the ESP32-S3 in place of the original ESP32, the ATGM336H-5N in place of the NEO-6M, the MPU6500 in place of the MPU6050, a managed lithium-ion supply in place of a buck converter from the vehicle battery, and the ADXL345 as an optional redundant impact channel. System design and architecture occupied September and concluded with the interim evaluation, from which the implementation phase is now planned.

CHAPTER 4

REFERENCES

- [1] H. Alai, R. Rajamani, and K. Sanjaya, (2024), “A Smart E-Scooter with Embedded Estimation of Rear Vehicle Trajectories for Rider Protection,” *Mechanical Systems and Signal Processing*, Elsevier.
- [2] G. Chhabra, et al., (2024), “Internet of Things Based Smart Framework for the Safe Driving Experience of Two Wheelers,” *Scientific Reports*, vol. 14, Nature Publishing Group.
- [3] “Development of Real-Time GPS Tracking and ETA on an Android Application Using ESP32 and u-blox NEO6MV2,” (2024), in *Proc. IEEE Int. Conf. on Computing, Communication and Networking Technologies (ICCCNT)*, IEEE.
- [4] Espressif Systems, (2024), *ESP32-S3 Technical Reference Manual*, Espressif Systems (Shanghai) Co., Ltd.
- [5] Espressif Systems, (2024), *ESP32-S3 Series Datasheet*, Espressif Systems (Shanghai) Co., Ltd.
- [6] InvenSense Inc., (2014), *MPU-6500 Product Specification*, Rev. 1.1, InvenSense Inc., San Jose, CA.
- [7] Analog Devices Inc., (2015), *ADXL345 Digital Accelerometer Data Sheet*, Rev. E, Analog Devices Inc., Norwood, MA.
- [8] Zhongke Microelectronics, *ATGM336H Series GNSS Module Data Sheet* (AT6558 multi-constellation positioning chip), Zhongke Micro (ZKW), Shanghai.
- [9] Bluetooth SIG, (2023), *Bluetooth Core Specification, Version 5.4*, Bluetooth Special Interest Group — Generic Attribute Profile (GATT).
- [10] National Marine Electronics Association, *NMEA 0183 Standard for Interfacing Marine Electronic Devices*, Version 4.11, NMEA.
- [11] Google, (2024), *Flutter Documentation*, <https://docs.flutter.dev>.
- [12] SQLite Consortium, (2024), *SQLite Documentation*, <https://www.sqlite.org/docs.html>.
- [13] Ministry of Road Transport and Highways, Government of India, *Road Accidents in India*, Annual Report, MoRTH, New Delhi.

- [14] United Nations, (2015), *Transforming Our World: The 2030 Agenda for Sustainable Development*, United Nations General Assembly Resolution A/RES/70/1 — Targets 3.6, 11.2.
- [15] R. W. Sinnott, (1984), “Virtues of the Haversine,” *Sky and Telescope*, vol. 68, no. 2, p. 159.